

**Context engineering**，就是在有限的上下文窗口里，**动态选择最有用、最少但足够的信息**，让 Agent 在多轮任务中持续做出正确决策。

通俗说：

过去我们关心的是“怎么把一句 prompt 写好”；现在做 Agent，更重要的是“每一步到底该把哪些背景、工具说明、历史记录、文件、搜索结果、示例放进模型脑子里”。

---

## 1. Context engineering 和 prompt engineering 的区别

文章认为，**context engineering 是 prompt engineering 的自然升级版**。

**Prompt engineering** 主要关注：

怎么写系统提示词、怎么组织指令、怎么让模型在一次任务里输出想要的结果。

**Context engineering** 关注的范围更大：

模型**每一次推理时，进入上下文窗口的所有 token**，包括系统提示词、用户消息、工具说明、MCP、外部数据、检索结果、历史对话、工具调用结果等。

通俗解释：

Prompt engineering 像是“写一份好说明书”。

Context engineering 像是“给一个员工安排完整工作台”：说明书、工具箱、历史资料、当前任务、可查资料、备忘录，哪些该放桌上，哪些该收起来，哪些该临时查。

文章第 3 页的图也表达了这个区别：单轮 prompt engineering 只是在固定上下文里塞入系统提示词和用户问题；而 Agent 的 context engineering **是在每一轮循环中**，从文档、工具、记忆、历史消息等大量可能信息里**挑选一部分放入上下文**。

---

## 2. 为什么上下文是稀缺资源

文章强调：**上下文窗口虽然越来越大，但不是越多越好**。

原因有三点：

第一，模型会出现 **context rot**。

也就是上下文越长，模型越容易漏掉、混淆或错误使用里面的信息。

第二，**Transformer 架构本身存在注意力压力**。

每个 token 都要和其他 token 建立关系，理论上会形成  $n^2$  级别的 token 关系。上下文越长，模型的注意力越分散。

第三，模型虽然能处理长上下文，但在长距离信息检索和推理上，精度可能下降。

通俗解释：

模型的上下文窗口就像人的办公桌。桌子再大，如果堆满了文件、聊天记录、日志、网页和工具输出，人也会找不到重点。

所以问题不是“能不能塞进去”，而是“塞进去之后模型还能不能抓住重点”。

---

## 3. 好的上下文应该是什么样

文章给出的总原则是：

**用最小的一组高信号 token，最大化模型完成目标行为的概率。**

这句话很关键。它不是说上下文一定要短，而是说上下文要“信息密度高”。

具体包括几个方面：

### 3.1 系统提示词要清晰，但不要过度硬编码

Anthropic 建议系统提示词要用简单、直接的语言，处在一个合适的“高度”。

两种常见错误：

一种是太具体，写成一堆脆弱的 if-else 规则（把逻辑一条条写死）。这样会导致维护困难，而且 Agent 一旦遇到没覆盖的情况就容易崩。

另一种是太抽象，比如“请表现得专业、智能、可靠”。这种话太泛，模型不知道具体该怎么做。

通俗解释：

好的系统提示词不是“把每一步都写死”，也不是“只喊口号”。

它应该像一个靠谱的工作规范：告诉模型**目标、边界、原则、输出要求**，但允许模型根据实际情况灵活判断。

第 5 页的图用一个横轴展示了这个问题：左边是过度具体、硬编码的提示词，右边是过度模糊的提示词，中间才是比较理想的状态。

### 3.2 Prompt 应该分区组织

文章建议把系统提示词分成清晰区块，比如：

```
<background_information>
<instructions>
# Tool guidance
# Output description
```

可以用 XML 标签，也可以用 Markdown 标题。重点不是格式本身，而是让信息边界清楚。

通俗解释：

这就像写一份项目文档：

背景是背景，任务是任务，工具说明是工具说明，输出格式是输出格式。

不要混在一起，否则模型容易误解优先级。

这点对你现在做 Agent 项目很有参考价值，尤其适合映射到你常说的：

**背景 → 指令 → 工具 → 输出格式 → 约束 → 示例**

---

## 4. 工具设计也是上下文工程的一部分

文章特别强调：**工具不是简单地“越多越好”。**

工具定义了 Agent 和外部世界交互的契约。好的工具应该：

**清晰、单一、鲁棒、输入参数明确、返回结果节省 token，并且尽量减少功能重叠。**

**常见失败模式是工具集过于臃肿。**

**如果人类工程师都说不清某个场景该用哪个工具，就不能指望 Agent 自己判断得更好。**

通俗解释：

工具就像员工能调用的内部系统。

如果你给他十几个名字相似、功能重叠、说明模糊的系统，他会浪费大量时间试错。

所以工具设计的关键不是“多”，而是“边界清楚、返回有用、调用成本低”。

---

## 5. 示例很重要，但不要堆满边界案例

文章认可 few-shot examples，也就是通过示例教模型怎么做。

但它反对把大量边界情况塞进 prompt。

更好的方式是挑选一组 **多样化、典型、标准的示例**，让模型从中学到行为模式。

通俗解释：

给模型示例，就像教新人做材料。

不需要把公司过去 100 个特殊案例都塞给他；更有效的是给他 3–5 个最典型、最标准、最能体现原则的样板。

---

## 6. Agent 应该“按需取上下文”，而不是一开始全塞进去

文章提出一个重要策略：**just-in-time context**。

意思是：不要在任务开始时把所有可能相关的信息都塞给模型，而是让 Agent 保留轻量级索引，比如文件路径、查询语句、网页链接、数据库表名，然后在需要时通过工具动态读取。

例如 Claude Code 不会一次性加载整个代码库，而是用 `glob`、`grep`、`head`、`tail` 等方式一点点查看需要的文件和片段。

通俗解释：

人做研究也不会把整座图书馆背下来。

我们会先知道“资料在哪里”，然后需要什么查什么。

Agent 也应该这样：**先有目录和索引**，再按需打开具体资料。

这对你的 Agent 项目非常重要。比如 Market Insight Agent 不应该一开始把所有市场数据、新闻、规则、历史报告全部塞进去，而应该让 Agent 根据异常项、资产类别、时间窗口去动态检索相关信息。

---

## 7. 预检索和自主检索可以结合

文章不是说所有信息都必须运行时再查。它认为很多场景下可以采用混合策略：

一部分核心信息提前放入上下文，保证速度和稳定性；

另一部分信息让 Agent 根据任务需要自主探索。

Claude Code 就是这种模式：

CLAUDE.md 会提前放进上下文，而代码文件则通过搜索和读取工具按需获取。

通俗解释：

就像开会前，你可以提前给参会者发会议目标、基本背景、关键规则；但具体证据、附件、历史邮件，不需要全部打印出来放桌上，需要时再查。

文章还提到，金融、法律这类内容变化相对没那么剧烈、但要求准确的场景，可能更适合混合策略。

---

## 8. 长周期任务需要特殊的上下文管理方法

对于持续几十分钟、几小时，甚至跨多轮会话的任务，单靠上下文窗口是不够的。文章提出三类方法：

## 8.1 Compaction：压缩上下文

当对话快到上下文上限时，把历史内容总结压缩，然后开启新的上下文窗口继续任务。

但压缩不能太激进，否则会丢掉之后才变得重要的细节。

通俗解释：

这像是项目中途写“阶段性纪要”。

不是把所有聊天记录原封不动保存，而是保留关键决策、未解决问题、重要文件、当前进展和下一步。

## 8.2 Structured note-taking：结构化笔记 / Agent 记忆

Agent 可以把重要进度写到上下文窗口外的文件或记忆里，比如 `NOTES.md`、`todo list`、项目状态文档等。之后需要时再读回来。

通俗解释：

这就是让 Agent 自己记工作日志。

不要求它一直记住所有细节，而是让它把关键状态沉淀成外部笔记。

这和你在项目里强调的 `AGENTS.md`、`CLAUDE.md`、`data_contract.md`、`pipeline.md` 很接近：这些文档本质上就是让 Agent 有稳定的外部上下文。

## 8.3 Sub-agent architecture：子 Agent 架构

复杂任务可以拆给多个子 Agent。

主 Agent 负责计划和综合，子 Agent 负责局部搜索、分析、执行，然后只把浓缩结果返回给主 Agent。

通俗解释：

这像是一个项目经理带几个分析师。

项目经理不用看每个分析师查过的所有资料，只需要看他们整理出的结论、证据和风险提示。

文章认为，这种方式尤其适合复杂研究和分析任务。

---

## 9. 三种长任务方法分别适合什么场景

文章给了一个简单判断：

**Compaction** 适合需要连续对话流的任务，比如长期 debug、持续写代码、反复沟通需求。

**Structured note-taking** 适合有明确阶段和里程碑的迭代开发，比如项目状态管理、todo、实验记录。

**Multi-agent** 适合复杂研究和分析，尤其是可以并行探索多个方向的任务。

通俗解释：

如果任务像“连续聊天”，用压缩。

如果任务像“长期项目”，用笔记。

如果任务像“复杂调研”，用多 Agent。

---

## 10. 启发

这篇文章最值得吸收的不是某个具体技术，而是一套设计原则：

第一，Agent 的核心不是“写一个很长的 prompt”，而是设计一套 **==上下文供给机制**。==

第二，文档不是普通说明，而是 Agent 的运行约束。比如 `AGENTS.md`、`CLAUDE.md`、`data_contract.md` 都是在定义 Agent 该如何理解任务、工具、边界和输出。

第三，工具设计要少而清晰。不要让 Agent 在一堆重叠工具里猜。

**第四，外部数据不要一次性全塞。应该通过文件路径、数据库查询、搜索工具、索引、摘要等方式按需读取。**

第五，长任务必须有“记忆机制”。可以是 compaction、todo、NOTES.md、状态文件，也可以是主 Agent + 子 Agent。